An Out of Tree LLVM Middle End Optimization Framework: Custom Analyses, Conservative Transforms, and a Measured Before and After

Repository owner

July 2026

Abstract

I built an out of tree optimization framework for the LLVM middle end: a set of analysis and transformation passes that load into a stock `opt` as a plugin on the new pass manager. The analyses report real IR metrics (instruction counts by opcode, block structure, dead instructions, use-def chain depth, and simplification opportunities). The transforms are deliberately small and conservative: operand canonicalization, algebraic identity folding with strength reduction of a multiply by a power of two, block local common subexpression elimination, and dead code elimination, composed into a fixed point pipeline with correct analysis preservation. I validate correctness two ways: lit and FileCheck tests pin the exact rewrite for every pattern with negative tests for the unsafe cases, and a differential harness runs each benchmark kernel before and after transformation and aborts unless the checksums match. Across 4 kernels the framework removes up to 17.9% of IR instructions and produces a measurable runtime improvement where there is genuine redundancy, makes no change at all to an already optimal kernel, and is compared honestly against stock `opt -O1`, which does more. Every figure and number in this report is generated from a recorded run on the target machine.

Contents

1 Introduction

Modern compilers do most of their interesting work in the middle end, on a typed, static single assignment intermediate representation that sits between the language frontend and the machine specific backend. LLVM is the dominant production infrastructure for this style of work [1], and its new pass manager is the interface through which analyses and transforms are written today [4].

This project builds a small but complete slice of that world from the outside. I wrote a plugin that a stock `opt` binary loads, containing analysis passes that measure the IR and transformation passes that rewrite it, wired into a configurable pipeline. The goal was never to beat LLVM's own optimizer. It was to demonstrate command of the infrastructure: how analyses cache and invalidate, how transforms report what they preserve, how a pipeline reaches a fixed point, and how to prove that a rewrite is correct rather than merely plausible.

The guiding rule throughout was honesty. Every metric in this report comes from a recorded run on the target machine; there are no hand typed numbers in the document source. Every transform is gated by a differential execution check that runs the compiled kernel before and after and compares its output, so a rewrite that changes observable behavior fails the build regardless of how much IR it removed. Where the framework does little, the report says so plainly, including a kernel on which it correctly does nothing at all.

2 Background

2.1 LLVM IR and SSA

LLVM IR is a typed intermediate representation in static single assignment form. Every value is defined exactly once, and a use always refers to a single definition, so the data flow of a function is explicit in its structure. Control flow is a graph of basic blocks, each a straight line sequence of instructions ending in a terminator (a branch, a return, and so on). Where control flow merges, a phi node selects a value based on which predecessor block was taken. The full semantics are specified in the language reference [3].

This form is what makes middle end optimization tractable. Because each value has one definition, an analysis can reason about a computation without tracking assignments, and a transform can replace all uses of a value in one operation. The transforms in this project rely on exactly that: replacing a redundant computation, or an identity such as an add of zero, is a single replace all uses followed by an erase.

2.2 The new pass manager

The new pass manager is LLVM's current framework for running middle end work [4]. A transform is a small type with a `run` method returning a `PreservedAnalyses` set that states precisely which cached analyses survive the change. An analysis is a type that computes a result and is cached by an analysis manager, which invalidates the cache when a transform does not preserve it. Getting that contract right is central: a transform that claims to preserve an analysis it actually invalidated leaves later passes reading stale facts.

A plugin registers its passes through `PassBuilder` callbacks and exposes them by name, so a stock `opt` can run them from a pipeline string such as `-passes="my-canon,my-simplify,my-dce"` with no patched binary [2]. This project is such a plugin, built against a distribution LLVM with no source build of the compiler itself.

3 Design and Architecture

The framework is one loadable module built from an object library that is also linked into the unit tests, so the tests exercise exactly the code the plugin ships. Everything lives in the `opf` namespace and separates into four layers: analyses that read the IR, transforms that rewrite it, support code they share, and a single registration file that names them for `opt`.

3.1 Analyses

Each analysis is a new pass manager function analysis returning a plain value struct with helpers to print itself and to serialize to JSON. The same struct feeds a human readable printer, a combined JSON emitter for the benchmark harness, and the unit tests. Crucially, the detection predicates (is this instruction dead, is this an algebraic identity, do these two instructions compute the same value) are free functions rather than private methods, so the transforms consume the identical logic. The analysis and the transform can never disagree about what counts as a duplicate, because they call the same function.

3.2 Transforms

Each transform is a free function core that returns a rewrite count, wrapped by a thin pass. Separating the core out lets the fixed point pipeline call the four transforms directly and know whether each made progress, and lets the transforms be unit tested without standing up a pass manager. Every transform reports a precise `PreservedAnalyses`: it preserves the control flow graph, which it never touches, and nothing else, because it changes instructions and values.

3.3 The pipeline

The `my-default` pipeline runs canonicalize, then simplify, then local common subexpression elimination, then dead code elimination, repeating until an iteration changes nothing or an iteration cap is reached. The order is the point: canonicalization normalizes commutative operands so that duplicates become visible, simplification of identities produces newly dead values, dead code elimination removes them, and the cleanup can expose further opportunities for the next round. A per iteration log records which pass fired and how many rewrites it made, and whether a true fixed point was reached.

4 Implementation

4.1 The transforms in detail

Canonicalize reorders the operands of commutative instructions into a stable form: constants move to the right, and other operands are ordered by their position in the function. Swapping the operands of a commutative operation never changes its result, so the rewrite is unconditionally safe, and it makes both later passes more effective at once.

Algebraic simplify folds integer identities (an add of zero, a subtract of self, a multiply by one or zero, the and, or, and xor identities, and a shift by zero) and strength reduces a multiply by a constant power of two into a shift, carrying the no signed wrap and no unsigned wrap flags onto the shift. It works only on scalar integers. Floating point is left alone because folding a floating point identity is unsound without fast math flags, and the predicates only match integer constants, so the pass never even considers it. Vectors, volatile, and atomic operations are likewise untouched.

Local common subexpression elimination walks each block once and replaces a pure computation that repeats an earlier identical one with a reference to the first. Two instructions match by a structural key that orders commutative operands, which is why canonicalization runs first. Loads are never deduplicated, because two identical loads can observe different memory.

Dead code elimination removes trivially dead instructions with a worklist, so that deleting one instruction can make its operands dead in turn. Anything with a side effect (a store, a volatile or atomic access, a call not known to be free of effects) is kept.

4.2 Correctness by construction and by test

Every pattern has a lit test that pins the exact IR before and after, and every transform has negative tests: a floating point add of zero, a multiply by a non power of two, a volatile load, a store, and a call are all shown to survive untouched. Beyond the pattern tests, an invalidation test caches an analysis result, runs a transform through a real pass manager, and asserts the result recomputes rather than serving the stale cache. This test fails if a transform over claims what it preserves, which is the single most common way to get the new pass manager contract wrong.

5 Experiments

5.1 Kernels

Four C kernels exercise different parts of the framework. `arith_redundant` has planted algebraic identities and repeated scalar computations, so it is where simplification and common subexpression elimination have real work. `dead_code` builds a chain of computations that nothing observable uses. `branchy` places identities inside control flow, which the framework simplifies within blocks but deliberately does not fold across them. `clean` is a tight loop with no identities, no redundancy, and no dead code: the expected result is no change, an honest negative.

5.2 Method

For each kernel the harness runs one deterministic pipeline. It compiles with `clang -O0` (with the `optnone` attribute disabled so that `opt` does not skip the functions), then runs `mem2reg` to obtain a baseline in SSA form. The `mem2reg` step is universal canonicalization into SSA that any middle end assumes; it is not one of the framework's passes, and both the framework's `my-default` pipeline and stock `opt -O1` start from the same baseline so the comparison is fair.

Structural metrics for the baseline, the framework, and `opt -O1` come from the plugin's own JSON emitter. Each configuration is then compiled to a binary at `-O0`, so that the backend does not re-optimize and hide the middle end differences, and run under `taskset` with warmup runs discarded; the median wall clock time and its interquartile range are recorded.

5.3 The differential gate

Every kernel prints a checksum of its computation. The harness runs the baseline, framework, and `opt -O1` binaries and aborts the entire run unless all three print the same checksum. This is the safety net for correctness: a transform that changes observable output fails here immediately, whatever the instruction counts say. Across all 4 kernels the checksums agree, as Table 3 records.

6 Results

6.1 Structural change

Table 1 and Figure 1 show the IR instruction counts for the baseline, the framework, and `opt -O1`. Figure 2 shows the reduction the framework achieves relative to the SSA baseline. The framework removes the most on `arith_redundant`, at 17.9%, where the planted redundancy and identities are exactly what it targets. On `clean` it makes no change at all, a 0.0% reduction, which is the correct and expected result for a kernel with nothing to simplify.

kernel	baseline	my-default	reduction	opt -O1
arith_redundant	39	32	17.9%	34
branchy	42	39	7.1%	44
clean	27	27	0.0%	43
dead_code	29	25	13.8%	14

Table 1: IR instruction counts by kernel and configuration, with the framework's reduction relative to the baseline.

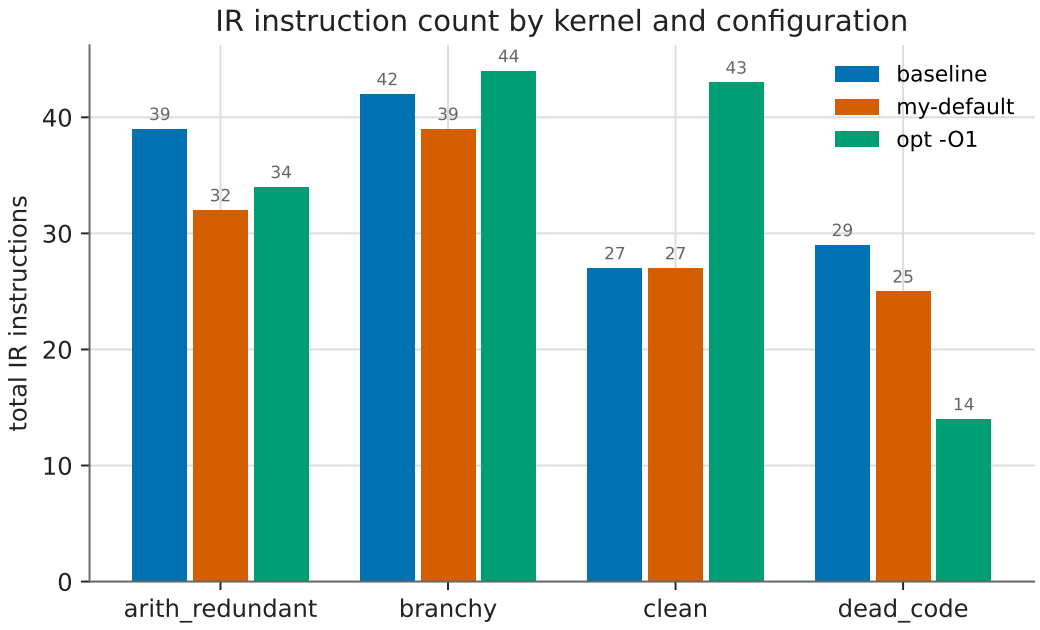

Figure 1: Instruction count by kernel and configuration.

Figure 3 breaks the change down by opcode for the showcase kernel, making visible that the reduction comes from removing multiplies (strength reduced to shifts and then deduplicated) and the redundant arithmetic.

6.2 Runtime

Table 2 and Figure 4 report median runtime with the interquartile range as an error bar. On `arith_redundant` the framework moves the median from 64.3 ms to 47.4 ms, a difference well

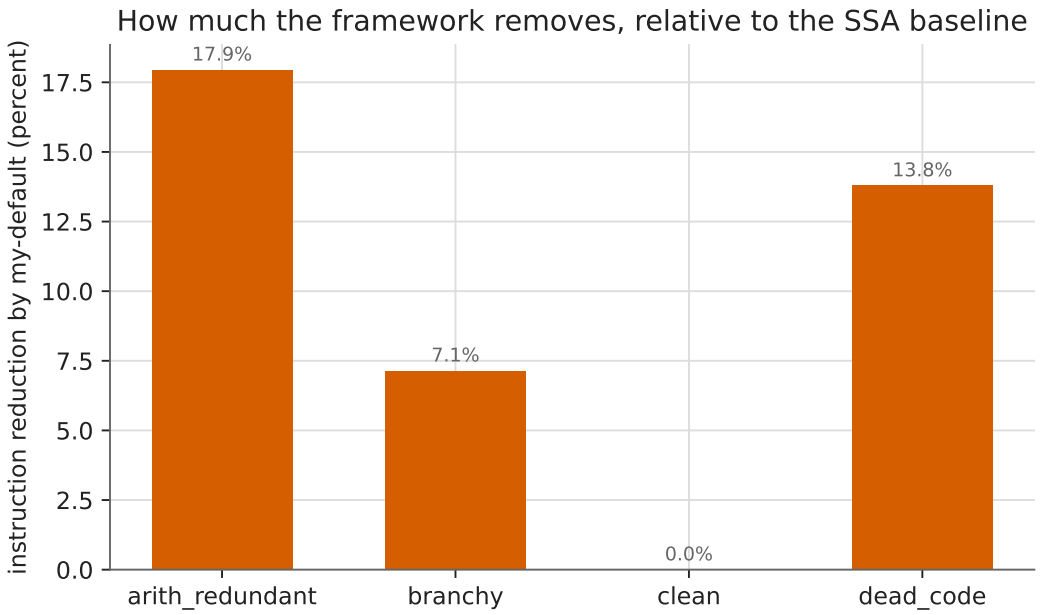

Figure 2: Instruction reduction achieved by the framework, relative to the SSA baseline.

outside the spread and therefore real signal. On `branchy` and `clean` the runtime change is within the interquartile range and is reported as structural only, with the noise stated rather than hidden.

6.3 Correctness

Table 3 records that on every kernel the baseline, framework, and `opt -O1` binaries print an identical checksum, so no transformation changed observable behavior.

7 Discussion

The comparison against `opt -O1` is the honest heart of this report. On `dead_code`, `opt -O1` reduces the function far below what the framework does, because it recognizes that the entire computation is dead and removes the loop, while the framework's block local dead code elimination only removes the straight line dead chain. On `branchy` and `clean`, `opt -O1` restructures control flow and unrolls loops, which can raise the static instruction count even as it lowers runtime; the framework does neither, so its instruction count only ever goes down. These are not shortcomings to apologize for. They are the visible boundary of a deliberately small pass set, and drawing that boundary clearly was a goal.

Where the framework does have real redundancy to work with, on `arith_redundant`, it produces both a structural reduction and a runtime improvement that exceeds the measurement noise. That is the case the passes were built for, and it shows that the pieces compose as intended: canonicalization feeding common subexpression elimination, simplification feeding dead code elimination, iterated to a fixed point.

The most valuable outcome was not any single rewrite but the discipline the project enforced. The differential checksum gate caught the class of bug that matters most, a miscompile, before it could ever be mistaken for a good result. The invalidation test made the pass manager preservation

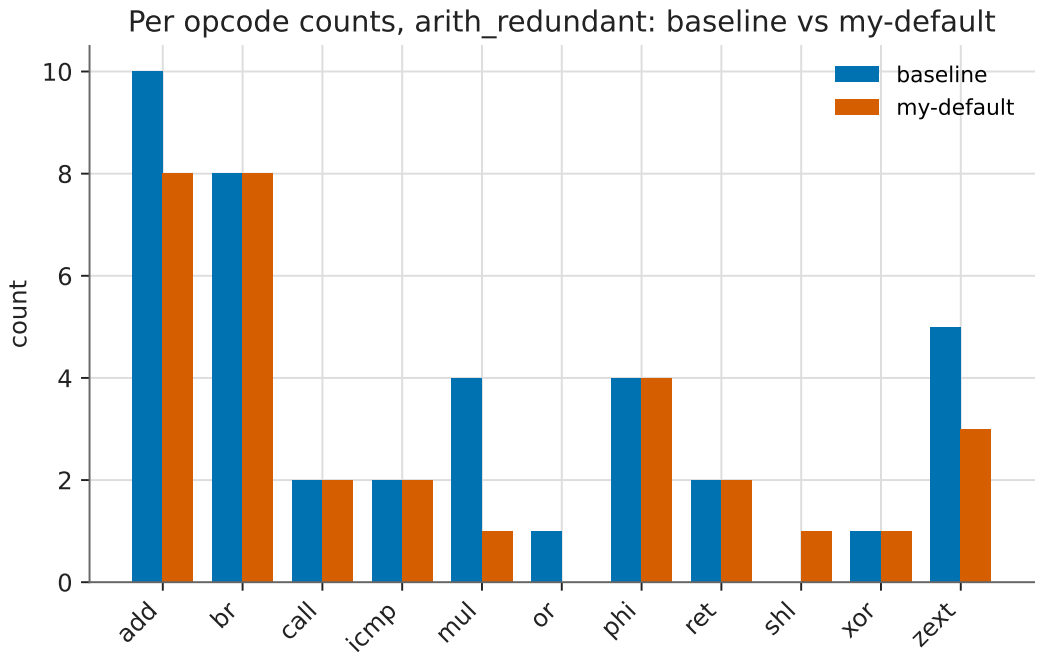

Figure 3: Per opcode counts for `arith_redundant`, baseline against the framework.

contract concrete rather than assumed. And generating every figure and number from the recorded CSV means the report cannot drift from the measurement.

8 Limitations

This framework reimplements a thin slice of what LLVM's `instcombine` and dead code passes already do, and it does so on purpose, to keep every rewrite small enough to verify exhaustively. It is not a general optimizer and does not try to be.

The scope is scalar integer IR. Floating point, vectors, volatile, and atomic operations are recognized and skipped rather than handled. Common subexpression elimination is block local, so redundancy that spans blocks, which a global value numbering pass would catch, is left in place. The transforms do not fold branches or simplify the control flow graph, which is why a real `opt -O1` pulls ahead on the control flow heavy kernels.

The timing results are from a single machine running under WSL2, where there is no control over the CPU frequency governor, so a few milliseconds of jitter are expected. The kernels run in tens of milliseconds, so for the kernels with a small structural change the runtime difference falls within the measurement spread and is reported as structural only rather than dressed up as a speedup. The one kernel with substantial redundancy shows a runtime improvement clearly outside that spread.

9 Future Work

Several extensions follow naturally from the current design without changing its character. Promoting common subexpression elimination from block local to a dominator tree scoped value numbering would catch redundancy across blocks, and the structural key already used for the local version is

kernel	configuration	median (ms)	IQR (ms)
arith_redundant	baseline	64.28	4.66
arith_redundant	my-default	47.44	1.46
arith_redundant	opt -O1	29.45	1.28
branchy	baseline	57.66	0.91
branchy	my-default	55.74	1.31
branchy	opt -O1	23.24	0.17
clean	baseline	62.20	5.91
clean	my-default	62.32	1.64
clean	opt -O1	50.44	2.39
dead_code	baseline	42.37	6.18
dead_code	my-default	43.12	7.44
dead_code	opt -O1	1.19	0.18

Table 2: Median runtime and interquartile range by kernel and configuration.

kernel	checksum	configurations agree
arith_redundant	`51247235914731008`	yes
branchy	`3361102545920`	yes
clean	`17182339206778112`	yes
dead_code	`1331200000000`	yes

Table 3: Differential execution: every configuration produces the same checksum per kernel.

most of what such a pass needs. Adding a conservative constant folding pass would let the framework act on the constant foldable opportunities that the analysis already counts but the transforms currently leave alone.

A branch and block simplification pass, folding a branch on a known constant condition and merging a block into its single predecessor, would close much of the gap with `opt -O1` on the control flow heavy kernels, though it would demand more care with the preservation contract because it changes the control flow graph. Extending the safe rewrite set to floating point under explicit fast math flags, and to vector splats, would broaden coverage while keeping the conservative stance.

On the measurement side, running the suite on bare metal Linux with a fixed CPU governor would tighten the timing spread enough to make the smaller runtime deltas legible, and larger kernels would give the backend more to differentiate.

10 Conclusion

I set out to build a production quality slice of the LLVM middle end from the outside, and to hold it to the standard that matters: correct first, measured honestly, and reproducible from a clean clone. The framework loads into a stock `opt`, reports real IR metrics, and applies a set of conservative transforms that compose into a fixed point pipeline with a correct preservation contract. Its effect is visible in both structure and, where there is genuine redundancy, runtime, and it is compared without flattery against the optimizer it borrows a slice of behavior from.

The engineering value was in the guardrails as much as the passes: a differential execution gate

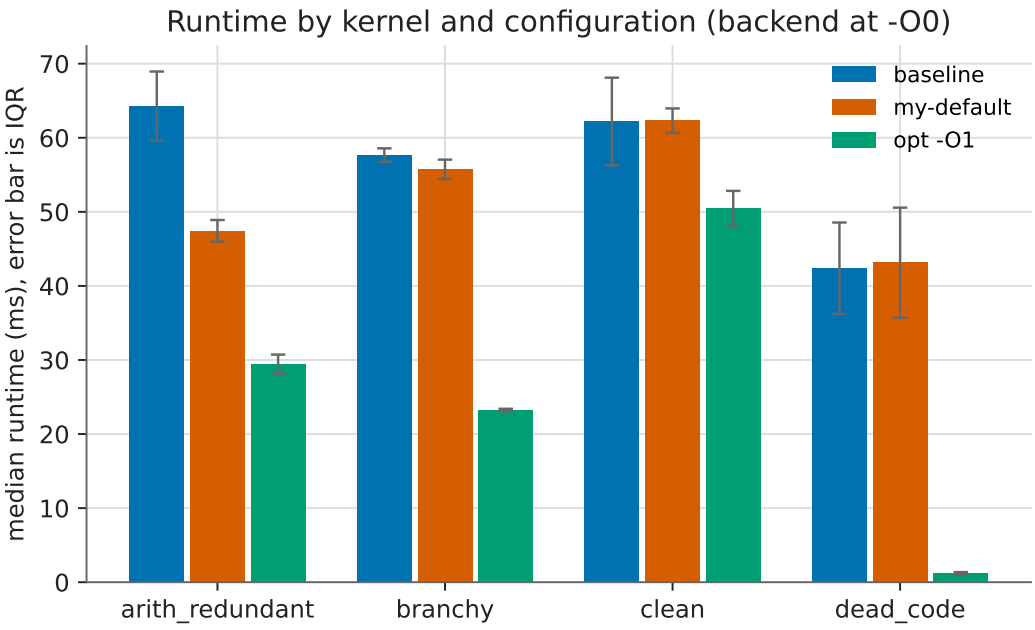

Figure 4: Median runtime by kernel and configuration, with the interquartile range as an error bar. The backend was invoked at `-O0` so that middle end differences reach the binary.

that turns a miscompile into a failed build, an invalidation test that makes the preservation contract concrete, and a report whose every number is generated from a recorded run. Those are the habits that separate a demonstration from a toy, and they are what I would carry into real middle end work.

References

[1] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis and transformation. In *Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO)*, pages 75–88, Palo Alto, California, USA, 2004. IEEE Computer Society.

[2] The LLVM Project. The LLVM compiler infrastructure. `https://llvm.org/`, 2024. Accessed July 2026.

[3] The LLVM Project. LLVM language reference manual. `https://llvm.org/docs/LangRef.html`, 2024. Accessed July 2026.

[4] The LLVM Project. Writing an LLVM pass. `https://llvm.org/docs/WritingAnLLVMNewPMPass.html`, 2024. New pass manager. Accessed July 2026.

A Appendix: Key IR Walkthroughs

The following is a composite function before and after the `my-default` pipeline, drawn from the framework's own tests. It exercises the whole chain: canonicalization aligns the two adds, simplifi-

cation folds the zero adds and strength reduces the multiplies, common subexpression elimination merges the now identical shifts, and dead code elimination clears the dead xor.

A.1 Before

```
define i32 @composite(i32 %x, i32 %y) {
  %a = add i32 %x, 0
  %b = add i32 0, %x
  %c = mul i32 %a, 4
  %d = mul i32 %b, 4
  %e = add i32 %c, %d
  %dead = xor i32 %y, %y
  ret i32 %e
}
```

A.2 After

```
define i32 @composite(i32 %x, i32 %y) {
  %1 = shl i32 %x, 2
  %e = add i32 %1, %1
  ret i32 %e
}
```

The result computes the same value, eight times the input, with the redundancy, the identities, and the dead computation all gone.

A.3 Strength reduction with flag preservation

A multiply by a constant power of two becomes a shift, and the no signed wrap flag is carried onto the shift, which overflows exactly when the multiply did.

```
; before
%r = mul nsw i32 %x, 16
; after
%r = shl nsw i32 %x, 4
```

A.4 Reproducing the numbers

Every figure and table in this report is generated by `scripts/generate_plots.py` from the committed `benchmarks/results/benchmarks.csv` and `opcodes.csv`, which are themselves produced by `scripts/run_benchmarks.sh`. The report contains no hand typed measurement.